ENGINEERING DOCUMENTATION / 01

Setup Instructions

Install, run, verify, and review the backend.

A practical guide for an evaluator or developer starting from a fresh checkout. It explains the shortest Docker path, the local Python alternative, environment settings, migrations, test isolation, and the Postman workflow.

Document reference	Value
Project	Expense Sharing App / Fairshare UI
Edition	Final recruiter submission
Prepared	16 October 2026
Application baseline	72f1182
API / schema	0.1.0 / Alembic head 0002
Currency / default timezone	INR / Asia/Kolkata

Inside this guide

- Verified Render demo and recruiter walkthrough
- Docker and local Python setup
- Render configuration, migrations and troubleshooting
- Tests, Postman and final handoff

Verified Render demo: https://expense-sharing-app-inda.onrender.com/

Source repository: https://github.com/manansethi1205/expense-sharing-app

Live UI and API verified on 16 October 2026. Free hosting can sleep; the free database expires 30 days after creation. Shared synthetic data only.

Examples use synthetic data. IDs shown in worked examples are illustrative; use IDs returned by your own requests.

Contents

1. Review the live Render demo 3

2. Choose a local setup path 5

3. Run the Docker stack 6

4. Develop with local Python 7

5. Configure and migrate safely 8

6. Verify behavior and use Postman 9

7. Run tests and quality checks 10

8. Troubleshoot common problems 11

9. Deploy or maintain the Render service 12

1. Review the live Render demo

No local installation is needed to review the deployed application. The Fairshare browser UI calls the real FastAPI endpoints and displays saved PostgreSQL data.

Resource	Verified URL
Browser demo	https://expense-sharing-app-inda.onrender.com/
Swagger UI	https://expense-sharing-app-inda.onrender.com/docs
OpenAPI contract	https://expense-sharing-app-inda.onrender.com/openapi.json
Database readiness	https://expense-sharing-app-inda.onrender.com/health/ready
Public source	https://github.com/manansethi1205/expense-sharing-app

Five-minute review

1. Open the demo and select Goa Trip (demo). If the free service is waking up, allow roughly a minute before retrying.

2. Review total spending and switch the personal-view selector between Mohit, Rahul and Ankit. Compare a person's allocated share with bills they actually paid.

3. Open Expenses and inspect View split. Category inference is explainable and can be corrected manually.

4. Choose Create a fresh demo trip before experimenting. It creates a separate group through normal API calls, preserving other reviewers' trip history.

5. In the fresh group, add an expense or try custom shares, then open Settle up and record a synthetic partial repayment. Refreshing the view shows the new debt and payment history.

Expected seeded scenario

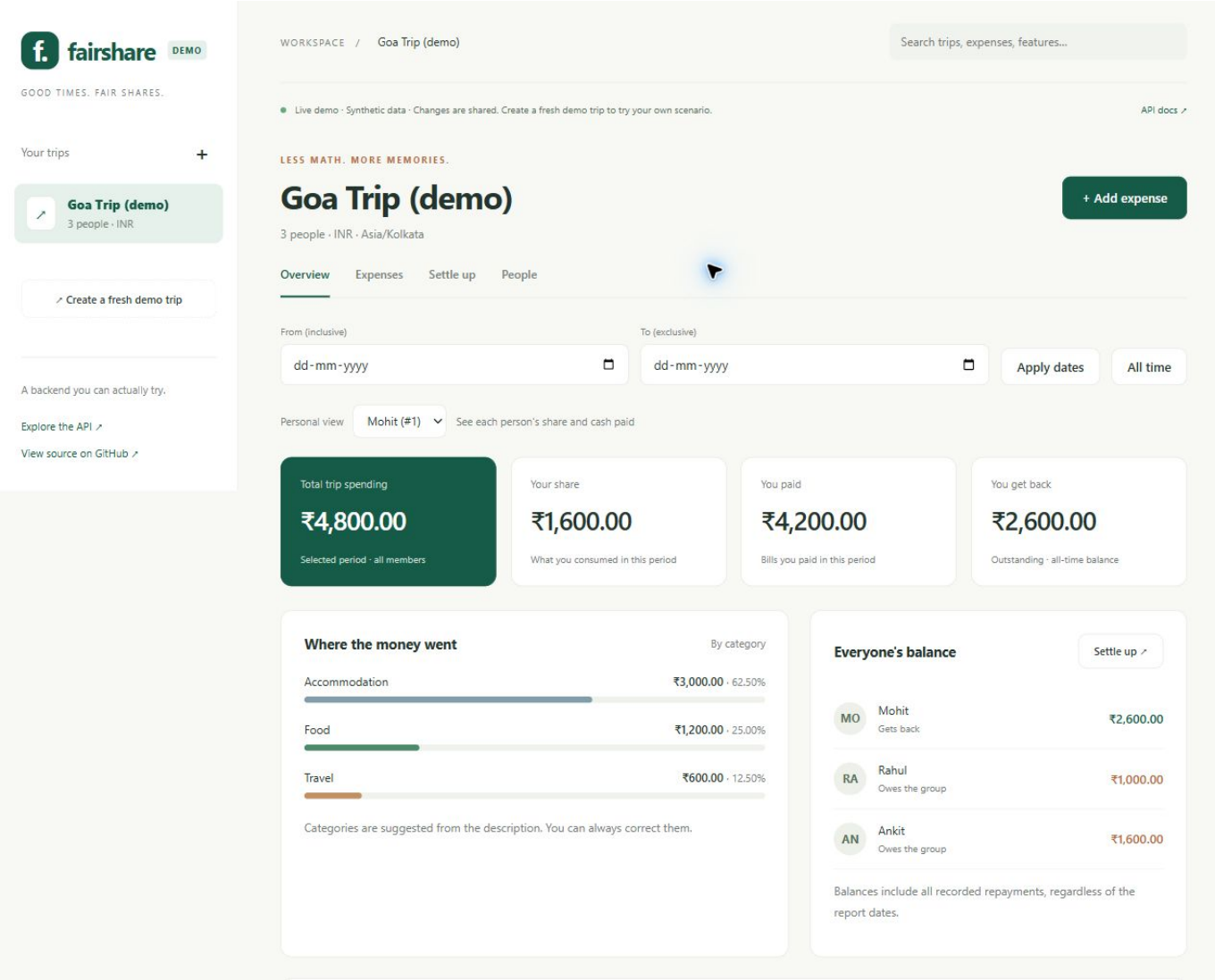
Bill	Paid by	Amount (INR)
Dinner	Mohit	1,200.00
Taxi to beach	Rahul	600.00
Hotel stay	Mohit	3,000.00

Total spending is INR 4,800.00, with INR 1,600.00 allocated to each member. Mohit paid INR 4,200.00 and receives INR 2,600.00. Rahul owes INR 1,000.00; Ankit owes INR 1,600.00. These are starting-state expectations; shared reviewer changes may alter an existing group. A fresh demo trip recreates the initial scenario.

> **Demo scope and lifetime**
>
> The demo has no authentication and contains synthetic shared data. Repayments are records, not real money movement. The free web service can sleep after inactivity, and the free PostgreSQL database expires 30 days after its creation. These hosting limits do not affect running the project locally.

Live dashboard: actual saved results

This screenshot was captured from the deployed Render application on 16 October 2026. It shows the seeded trip after migrations and sample data were verified.



Mohit's personal view shows INR 1,600.00 consumption, INR 4,200.00 bills paid and INR 2,600.00 receivable. The group category totals and all three net positions come from the same saved expense/share/settlement ledger. The screenshot captures a starting state; later shared reviewer activity may change the live display.

2. Choose a local setup path

Use Docker for the quickest reproducible review. It runs PostgreSQL, a one-shot migration container, and a nonroot API container. Use local Python when editing or debugging application code; PostgreSQL is still required.

Path	Prerequisites	Best use
Docker	Git; Docker Engine/Desktop running; Compose v2; internet for first build	Review the complete service without installing Python dependencies on the host
Local Python	Git; Python 3.12+; uv; PostgreSQL 17 or the supplied Docker DB	Edit code, use reload, run tests and migration tools

Repository checkout

```
git clone https://github.com/manansethi1205/expense-sharing-app.git
cd expense-sharing-app
```

All commands in this guide run from that repository directory. Commands contain no prompt prefix, so they can be copied directly. Docker must be available to the same operating-system user executing the commands.

What you receive

Location	Purpose
app/	FastAPI routes, validation, business services, SQL reports and pure accounting functions
migrations/	Alembic schema revisions 0001 and 0002
tests/	Unit tests and real PostgreSQL integration tests
docs/	Postman collection, plain HTTP walkthrough and verification record
README.md / decisions.md / flow.md	Project overview, reasoning/tradeoffs and actual code call flows

Assignment scope

This is an INR-only backend demo without authentication. Use synthetic data. The service records external cash/UPI/bank payments; it does not send money or verify bank receipts.

3. Run the Docker stack

Start the services and wait for readiness

```
docker compose up --build -d --wait --wait-timeout 120
docker compose ps
```

The first build downloads the Python base image and locked packages. Later builds reuse unchanged layers. The database health check must pass before migrations run; the API starts only after migration completion.

Service	Expected state	Meaning
db	Running / healthy	PostgreSQL is accepting connections
migrate	Exited with code 0	alembic upgrade head completed; exit is expected
api	Running / healthy	The API can query its database

Open the review tools

URL	Purpose
http://localhost:8000/docs	Swagger UI: inspect contracts and execute requests
http://localhost:8000/redoc	Alternative human-readable API reference
http://localhost:8000/openapi.json	Machine-readable OpenAPI contract
http://localhost:8000/health/live	Process liveness; no database query
http://localhost:8000/health/ready	Database connectivity check; 503 if unavailable

Seed synthetic trip data

```
docker compose exec api python -m scripts.seed_demo
```

The first seed run returns actual IDs for Mohit, Rahul, Ankit and Goa Trip (demo). It records Dinner, Taxi to beach, and Hotel stay, so it demonstrates multiple payers and categories. Running it again returns the existing demo group's ID rather than duplicating it. These three expenses mean its balances differ from the single-dinner example in the API guide.

> **Ports and persistence**
>
> Published ports bind to localhost: API 8000, database 5432. A named volume preserves application data when containers stop. docker compose down stops/removes the project containers while keeping that volume; it is not a database reset.

4. Develop with local Python

Install Python 3.12+ and uv, then create the locked project environment:

```
uv sync --locked
```

Create a local environment file

PowerShell:

```
Copy-Item .env.example .env
```

macOS/Linux or a POSIX shell:

```
cp .env.example .env
```

Keep .env local; it is ignored by Git. Start only the supplied database service, or point DATABASE_URL at a separately created development PostgreSQL database:

```
docker compose up -d --wait db
uv run alembic upgrade head
uv run python -m scripts.seed_demo
uv run uvicorn app.main:app --reload
```

Expected result

Uvicorn listens on localhost:8000 and reloads when Python source changes. The database schema is created by Alembic, not by importing the API. If both the container API and local API try to bind port 8000, stop the container API first:

```
docker compose stop api
```

When using an existing PostgreSQL installation, create a development database and role with permission to create/alter its schema. The supplied Compose configuration already creates the expense role and expenses database. The role/password in .env.example are demo credentials, not production configuration.

> **Host versus container addresses**
>
> Local Python connects to localhost and the published DB_PORT. Inside Compose, API/migrate connect to db:5432 through the private network. Compose supplies its own DATABASE_URL, so changing the host DATABASE_URL alone does not redirect the container API.

5. Configure and migrate safely

Setting	Default / accepted value	Effect
DATABASE_URL	postgresql+psycopg://expense:expense@localhost:5432/expenses	Local Python connection; supplied explicitly inside Compose
EXACT_SETTLEMENT_THRESHOLD	16; integer from 0 through 16	Maximum nonzero balances using exact optimization; 0 forces greedy for nonempty positions
API_PORT	8000	Host port mapped to API container port 8000
DB_PORT	5432	Host port mapped to PostgreSQL container port 5432
TEST_DB_PORT	5433	Host port for the separate disposable test database
TEST_DATABASE_URL	PostgreSQL URL ending in database name _test	Read by integration tests; must differ from the application URL

If DB_PORT or TEST_DB_PORT changes, update the corresponding host-side URL. API_PORT changes the host URL used by browsers, Postman and live smoke tests; it does not change the API container's internal port.

Migration commands

```
uv run alembic current
uv run alembic upgrade head
uv run alembic check
```

The current head is 0002. Revision 0001 creates the relational schema. Revision 0002 removes the old per-expense settlement cap, allowing one payment to clear several individually valid expenses. alembic check compares the current database to the ORM models; it should report no new upgrade operations.

Future schema changes

```
uv run alembic revision --autogenerate -m "describe the schema change"
```

Review generated SQL operations before applying them. A downgrade from 0002 to 0001 refuses to proceed if payments above the old cap already exist. This is deliberate: the migration never truncates or rewrites financial history to make a downgrade fit.

6. Verify behavior and use Postman

Run the complete collection

Import docs/expense-sharing.postman_collection.json and either Local.postman_environment.json or Render.postman_environment.json. Select the environment before running; its base_url overrides the collection's localhost default. Render uses https://expense-sharing-app-inda.onrender.com/api/v1. No authorization header is needed. Run requests in order in the Collection Runner; captured IDs avoid assumptions about existing seed data.

Stage	What the collection proves
Create users/group/members	IDs are captured into collection variables; no assumption about an empty database
Equal dinner	A 1200.00 expense allocates three 400.00 shares
Balances and analytics	Two 400.00 debts; Mohit paid 1200.00 but consumed 400.00
Settlements	Two recorded payments clear the group
Custom split and category	500.00 / 300.00 / 200.00 allocation and manual category correction
Search and drill-down	Entity search, feature discovery and paginated consumption records

The collection contains 21 requests with response/status checks. Re-running it creates a fresh set of synthetic entities and uses their returned IDs. The plain docs/demo.http alternative includes every request too, but its fixed IDs assume an empty database.

Live HTTP smoke check

```
uv run python -m scripts.smoke_http --base-url http://localhost:8000
```

The smoke script creates synthetic records, exercises the original dinner scenario, verifies personal analytics, records the suggested payments, and checks the final settled state. It leaves those records for inspection; run it against a demo/disposable instance.

A useful manual acceptance check

Create a group with three members, record Dinner 1200.00 paid by Mohit with split_type omitted, and verify Rahul and Ankit each owe Mohit 400.00. Record Rahul's payment, verify only Ankit's debt remains, then record Ankit's payment and verify is_settled=true.

7. Run tests and quality checks

Unit checks require no database

```
uv run pytest tests/unit -q
uv run ruff check .
uv run ruff format --check .
```

The full suite uses a dedicated PostgreSQL instance

```
docker compose --profile test up -d --wait test-db
uv run --env-file .env pytest -q
```

The .env example includes TEST_DATABASE_URL for localhost:5433/expenses_test. uv run --env-file loads that value into the test process. Without TEST_DATABASE_URL, integration tests explicitly skip; a green unit-only run is not evidence that the entire API/database suite ran.

> **Database isolation**
>
> The test URL must name a PostgreSQL database ending in _test and differ from the application's URL. Every integration case creates and migrates a generated schema, then drops only that schema. The role needs CREATE SCHEMA privileges. Tests do not reset the application's public schema or application data.

What is covered

The final baseline has 87 passing tests: 54 unit cases and 33 PostgreSQL integration tests. Coverage includes exact paise allocation, custom-split validation, membership snapshots, balances, exact optimizer versus an independent oracle, cycle cancellation, category context, search, migration safety, concurrent payments, snapshot reports, hosted URL normalization and static asset serving. The final rerun completed in 24.26 seconds; timings depend on the machine.

Optional measurement and shutdown

```
uv run python -m scripts.benchmark_settlements
docker compose --profile test stop test-db
docker compose down
```

Benchmark results are written to an ignored JSON file. Sample local measurements support the 16-member exact threshold, but are not a worst-case latency guarantee. GitHub Actions also runs lint, format, full PostgreSQL tests, Compose validation and a Docker build.

8. Troubleshoot common problems

Symptom	Likely cause / next action
Cannot connect to Docker	Start Docker Desktop/Engine; confirm docker version works for the current user. Compose v2 is required.
Port is already allocated	Stop the other API/DB or set API_PORT/DB_PORT/TEST_DB_PORT in .env. Update host-side URLs to match.
API returns 503 readiness	Check database availability and connection settings; readiness tests connectivity, not migration revision.
relation/table does not exist	Run alembic upgrade head against the same DATABASE_URL used by the API. The application does not auto-create tables.
migrate container is exited	Exit code 0 is expected for the one-shot migration job. Nonzero requires inspecting its logs.
422 on expense or payment	Check amount precision/limits, null text, body field names, strict integer IDs and timezone-aware timestamps.
409 on payment	The payer/recipient/mode does not match current debt, the group is already net-settled, or another payment changed the state. Fetch a new preview.
Tests pass but API tests skip	Set TEST_DATABASE_URL and start test-db; confirm it ends in _test and is distinct from the app DB.
Postman IDs are unresolved	Run the collection from its first request so capture scripts set each variable; check base_url and prior response statuses.

Inspect service logs

```
docker compose logs --tail=100 api
docker compose logs --tail=100 migrate
docker compose logs --tail=100 db
```

Before handing over the assignment

Confirm the repository URL opens, a fresh checkout starts, the migration reaches 0002, Swagger is accessible, the complete collection runs, and all integration tests actually execute. Read the API and schema PDFs for accounting assumptions and API-specific explanations.

CI results | Detailed verification record

9. Deploy or maintain the Render service

The existing live service was configured manually in Oregon with PostgreSQL 18. Local Docker/tests and the reusable Blueprint use PostgreSQL 17; the Blueprint selects Singapore for new resources. Existing manually created services do not automatically inherit render.yaml. Keep the web service and its database in the same region.

Required service settings

Setting	Value / source
Runtime / instance	Docker / Free
Docker Command	python -m scripts.start_render
Health Check Path	/health/ready
DATABASE_URL	Internal Database URL from the Render PostgreSQL service; keep private
EXACT_SETTLEMENT_THRESHOLD	16
PORT	Supplied by Render; startup binds 0.0.0.0 to this value

No .env upload is needed on Render. Add the variables in the web service's Environment settings. Do not copy the localhost database URL from .env.example. API_PORT, DB_PORT, TEST_DB_PORT and TEST_DATABASE_URL belong to local development/testing.

```
python -m scripts.start_render
# upgrade head -> idempotent seed -> Uvicorn on PORT
```

Startup applies migrations before accepting traffic. The seed creates the synthetic trip only if absent and never resets reviewer changes. Running migrations at startup is a single-instance free-demo tradeoff; a production setup should use a dedicated migration stage.

Provisioning a separate copy

Use Deploy to Render or create a Blueprint from render.yaml. It defines a free web service and free PostgreSQL 17 in Singapore, links DATABASE_URL and blocks external database access. If the account already has its free database, resolve the quota rather than unintentionally choosing a paid plan.

Diagnose deployment failures

Observation	Action
Connection refused at localhost:5432	Set DATABASE_URL to Render's internal database connection URL, save and deploy.
Readiness 200 but relation groups does not exist	Database connectivity works, but migrations were skipped. Set Docker Command to python -m scripts.start_render and deploy.
UI reports HTTP 500 / 502	Read current Render logs. The UI handles non-JSON server/proxy failures; repeated retries do not fix schema or configuration errors.
Empty or changed demo history	Data is shared. Create a fresh demo trip; do not reset another reviewer's records.

After deployment, verify the root UI, /docs, /health/ready and a real group API. The current public demo passed the full HTTP scenario. Application baseline 72f1182 also passed GitHub CI.

Hosting references: Render free-tier limits and Docker settings. Free service idle wakeup is about a minute; free databases expire after 30 days.